UNIVERSIDAD INDUSTRIAL DE SANTANDER

Escuela de Ingeniería de Sistemas e Informática

MUEBLESYA

Portal web de venta de muebles con despacho a domicilio

Informe de la FASE-II: aplicativo con Node.js, Express, MySQL y asesor de IA

Integrantes

Nicolas Soto Sanchez — Scrum Master

Brandon Andres Jaimes Romero — Product Owner

Nicolas David Sanguino Ortiz — Development Team (Frontend)

Juan Sebastian Araujo Contreras — Development Team (Backend y BD)

Subgrupo 06 · Asignatura: Programación Web

Bucaramanga, 2026

Acceso al sitio, usuarios y credenciales

URL de invocación (desarrollo): `http://localhost:5173` (frontEnd con Vite) — API y contenidos: `http://localhost:3000`.
URL en modo servidor único: `http://localhost:3000` (tras ejecutar `npm run build` en frontEnd, Express sirve todo el sitio).
Repositorio GitHub: *(pegar aquí el enlace del repositorio del equipo)*

Usuarios del sitio (formato: usuario | clave // descripción)

usuario | clave	descripción
`Admin | 1234`	// Administrador: ve «Administración» (CRUD de todas las tablas) y todos los pedidos y archivos.
`Cliente | 1234`	// Cliente registrado: pedidos propios, mis archivos y asesor con historial.
`(visitante)`	// Sin login: catálogo, asesor IA, Bucaramanga, multimedia y contacto.

Usuario de la base de datos MySQL

`un_usr | una_clave` // base de datos `mueblesya` (se crea automáticamente).

Los usuarios se crean automáticamente la primera vez que se inicia el servidor; las claves se almacenan cifradas (hash scrypt). Un visitante puede registrarse y queda con rol *cliente*.

1. Objetivos de la aplicación

Objetivo general

Desarrollar la segunda fase de **MueblesYA**: conectar el frontend en React con un backend Node.js/Express y una base de datos MySQL, de modo que los visitantes consulten el catálogo y los usuarios registrados realicen pedidos, gestionen archivos y reciban recomendaciones de una inteligencia artificial, mientras el administrador mantiene todos los datos del sitio.

Objetivos específicos

- Organizar el proyecto en las carpetas frontEnd y backEnd, con rutas (router) y controladores separados.
- Crear automáticamente una base de datos MySQL con mínimo tres tablas y el usuario un_usr.
- Reemplazar el login simulado por autenticación real contra la base de datos, con vistas de visitante, registrado y administrador.
- Implementar un CRUD RESTful que permita incluir, modificar, eliminar, listar y buscar en todas las tablas.
- Permitir subir y descargar archivos de cualquier tipo entre cliente y servidor.
- Implementar un servidor de contenidos con express.static para imágenes turísticas de Bucaramanga y su área metropolitana.
- Integrar la API de Gemini como aporte de IA: un asesor que recomienda muebles según personas, mascotas, presupuesto y foto del espacio.

2. Primera página del sitio

Captura de la página de inicio en ejecución: barra de menús fija con las nuevas opciones «Asesor IA» y «Bucaramanga», título y carrusel; más abajo continúa el diseño a tres columnas, los productos destacados (ahora leídos de MySQL) y el pie de página.



Figura 1. Primera página del sitio MueblesYA (FASE-II).

3. Estructura jerárquica del sitio en disco

```
06_App/
├── README.txt
├── .gitignore
├── frontEnd/                      (FASE I mejorada: React + Vite)
│   ├── index.html · package.json · vite.config.js
│   ├── public/favicon.svg
│   └── src/
│       ├── main.jsx · App.jsx
│       ├── api/            api.js · useCategorias.js
│       ├── assets/images/  logo, banners, fotos del equipo (SVG)
│       ├── components/     NavBar · LoginModal · Footer · HeroCarousel
│       │                   ProductCard · PedidoModal · CrudTabla
│       ├── context/        AuthContext.jsx
│       ├── data/           team.js
│       ├── pages/          Home · Catalogo · Multimedia · Contacto
│       │                   Asesor · Bucaramanga · Panel · Pedidos
│       │                   Archivos · Admin · NotFound
│       ├── routes/         PrivateRoute.jsx
│       └── styles/         main.css
└── backEnd/                       (FASE II: Node.js + Express + MySQL)
    ├── server.js · package.json · .env.example · .gitignore
    ├── public/             (express.static  ->  /public)
    │   ├── media/          promo.mp4 · bienvenida.mp3
    │   ├── turismo/        3 imágenes de Bucaramanga y su área
    │   └── productos/      imágenes de categorías y productos
    ├── uploads/            archivos subidos por usuarios (privado)
    └── src/
        ├── app.js
        ├── config/         env.js
        ├── database/       connection.js · initDb.js
        ├── middleware/     auth.js · upload.js · errores.js
        ├── routes/         index · auth · catalogo · pedidos
        │                   archivos · ia · admin  (RUTAS)
        ├── controllers/    auth · catalogo · pedidos · contacto
        │                   archivos · ia · admin · crud.factory (CONTROLADORES)
        ├── services/       gemini.service.js
        └── utils/          security.js · http.js
```

Significado de carpetas y archivos

Elemento	Significado
frontEnd/	Todo el cliente (interfaz React). Se ejecuta con Vite en desarrollo y se compila a frontEnd/dist.
frontEnd/src/api	api.js: cliente HTTP (fetch + token). useCategorias.js: hook que consulta las categorías a la API.
frontEnd/src/components	Piezas reutilizables: barra de menús fija, modal de login/registro, pie de página, carrusel, tarjeta de producto, modal de pedido y la tabla CRUD genérica.
frontEnd/src/context	AuthContext: sesión real (token JWT) y rol del usuario; controla el modal de login.
frontEnd/src/pages	Una página por ruta. Públicas: Home, Catalogo, Asesor, Bucaramanga, Multimedia, Contacto. Privadas: Panel, Pedidos, Archivos; solo admin: Admin.
frontEnd/src/routes	PrivateRoute: guardián de rutas privadas y de rutas solo para administrador.
backEnd/server.js	Arranque: prepara la BD y levanta Express en el puerto 3000.
backEnd/.env.example	Plantilla de configuración (BD, secreto JWT y clave de Gemini). Se copia como .env, que no se sube a GitHub.

backEnd/public	Servidor de contenidos con express.static: video, audio, imágenes de productos y 3 imágenes turísticas.
backEnd/uploads	Archivos que suben los usuarios. No es público: solo se entrega mediante la API con permisos.
backEnd/src/routes	ROUTER: define las URL de la API y las asocia a un controlador (un archivo por funcionalidad).
backEnd/src/controllers	CONTROLADORES: lógica de cada ruta. crud.factory.js genera el CRUD de las tablas; ia.controller.js es el asesor de IA.
backEnd/src/database	connection.js: pool MySQL. initDb.js: crea BD, usuario un_usr, tablas y datos iniciales automáticamente.
backEnd/src/middleware	auth.js (sesión, roles, límite de peticiones), upload.js (multer), errores.js (respuestas de error JSON).
backEnd/src/services	gemini.service.js: conexión con la API REST de Google Gemini.
backEnd/src/utils	security.js: hash scrypt de contraseñas y tokens firmados HS256. http.js: errores HTTP controlados.

4. Usuarios y credenciales

Formato `usuario | password`:

usuario | password	descripción
`Admin | 1234`	Administrador: ve «Administración» (CRUD de todas las tablas) y todos los pedidos y archivos.
`Cliente | 1234`	Cliente registrado: pedidos propios, mis archivos y asesor con historial.

Base de datos: `un_usr | una_clave`.

5. Modelo relacional de la base de datos

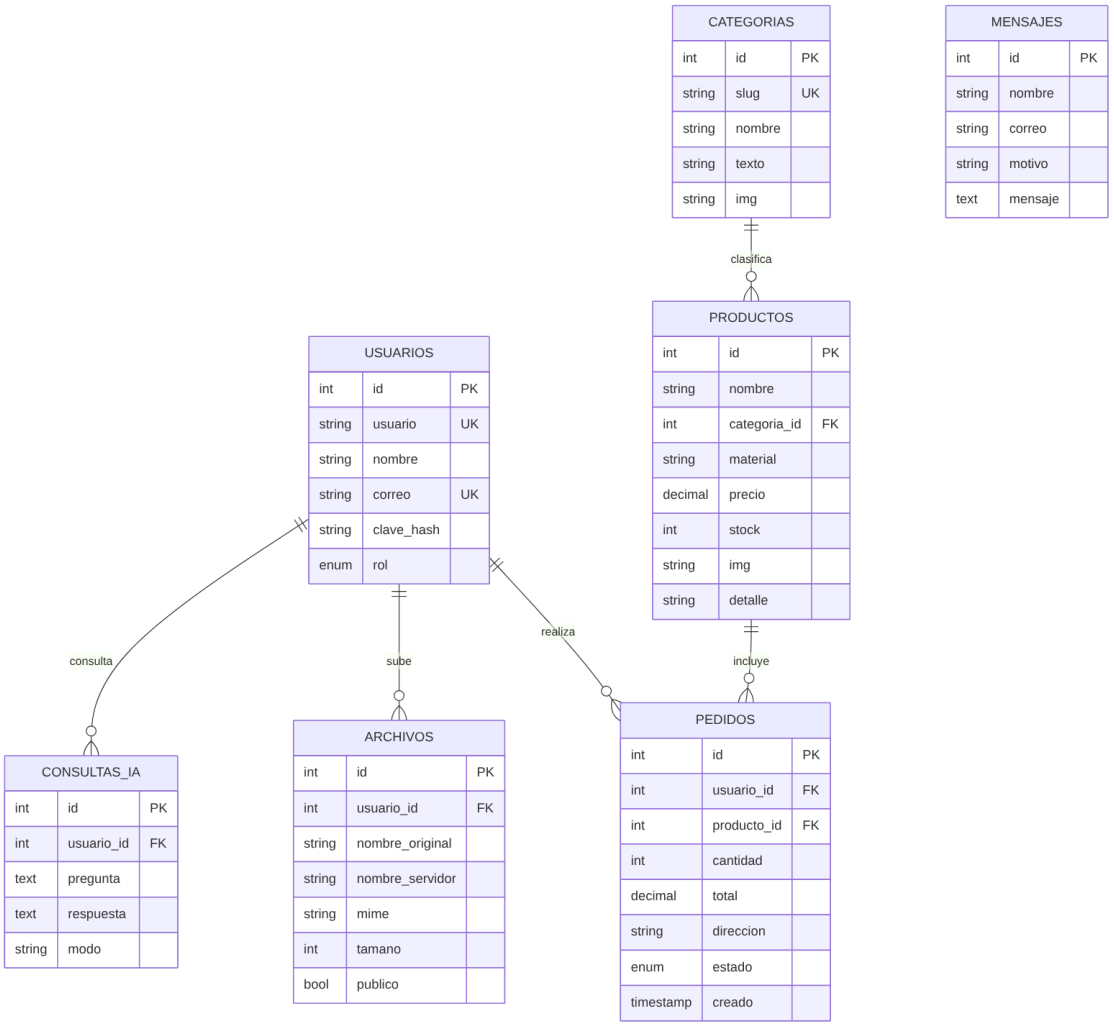


Figura 2. Modelo relacional de la base de datos «mueblesya» (7 tablas).

Cómo instalar o acceder a la base de datos

1. Abrir XAMPP e iniciar el módulo **MySQL** (puerto 3306).
2. **Automático (recomendado):** al ejecutar `npm start` en backEnd, el programa crea la base `mueblesya`, el usuario `un_usr` con clave `una_clave`, las 7 tablas y los datos iniciales. Para crear la BD y el usuario usa la cuenta `root` de XAMPP (sin clave); si tu root tiene clave, ponla en `DB_ADMIN_PASSWORD` del archivo `.env`.
3. **Manual desde phpMyAdmin** (pestaña SQL):

   ```
   CREATE DATABASE mueblesya CHARACTER SET utf8mb4;
   CREATE USER 'un_usr'@'localhost' IDENTIFIED BY 'una_clave';
   GRANT ALL PRIVILEGES ON mueblesya.* TO 'un_usr'@'localhost';
   ```

   Luego se inician el servidor y las tablas se crean solas.
4. Para consultar los datos: `http://localhost/phpmyadmin` → base `mueblesya`.

6. Listas de funcionalidades

Lista 1: funcionalidades del aplicativo web

- Diseño responsivo (celular, tablet y escritorio) con Bootstrap.
- Barra de menús fija con dropdown de categorías, popover de ayuda y submenús del panel.
- Portada con carrusel, tres columnas (acordeón | contenido | collapse y audio) y pie con contacto, redes y modal del equipo.
- Efectos: fade, modal, collapse, popover, dropdown, pestañas, letra capital, video y sonido.
- Catálogo con buscador y filtro por categoría (ahora desde MySQL).
- Formulario de contacto con validación (ahora guardado en BD).
- Rutas públicas, privadas y 404; vista de visitante y vista de usuario registrado.
- Seguimiento de pedidos del usuario.
- Administración de inventario (ahora CRUD completo).
- Página Multimedia con video y audio.
- Todas las de la Lista 2 (FASE-II).

Lista 2: nuevas funcionalidades implementadas en la FASE-II

Funcionalidad	Explicación
Backend REST	Node.js + Express con ROUTER (carpeta routes) y CONTROLADORES (carpeta controllers) separados.
Base de datos MySQL	7 tablas creadas automáticamente al iniciar el servidor, con datos de ejemplo y usuario un_usr / una_clave (XAMPP).
Login real y registro	Usuario y contraseña verificados en la BD; contraseñas con hash scrypt; sesión con token firmado; roles admin y cliente.
Vistas visitante / registrado / admin	El menú y las rutas cambian según el rol; /panel/admin exige rol admin tanto en el cliente como en el servidor.
CRUD RESTful de todas las tablas	Incluir, modificar, eliminar, listar y buscar en productos, categorías, usuarios, pedidos y mensajes (pestañas en Administración).
Pedidos reales	El cliente pide un producto; se valida el stock, se descuenta y se registra el total y la dirección.
Subir y bajar archivos	Cualquier tipo (texto, imagen, video, PDF…) hasta 25 MB desde «Mis archivos»; descarga con permisos; los marcados como públicos aparecen en Multimedia. Las imágenes de producto se suben desde el CRUD.
Servidor de contenidos	express.static sirve /public: video, audio, imágenes de productos y 3 imágenes turísticas de Bucaramanga, Girón y Floridablanca. Se corrige así la Multimedia de la Fase I.
Aporte IA: Asesor MueblesYA (Gemini)	El cliente indica personas, mascotas, presupuesto, ambiente y, opcionalmente, una foto de su espacio o de su mascota. El servidor consulta a la API de Gemini con el catálogo real y devuelve productos recomendados con su justificación y consejos. Valida los IDs contra la BD, guarda el historial y, si Gemini no responde, usa un modo básico por reglas.
Seguridad	Clave de Gemini solo en el servidor (.env), límites de peticiones, validación de datos, archivos subidos nunca se ejecutan en el navegador.

Lista 3: funcionalidades solicitadas pero no implementadas

Funcionalidad	Motivo o reemplazo
Recuperación de contraseña por correo	El enunciado indica no implementarla este semestre.
Módulo «Vista en tu espacio» de la Fase I	Se reemplaza por el Asesor IA con análisis de foto, que cubre la necesidad (orientar según el espacio) sin requerir realidad aumentada.
reCAPTCHA en el registro	Se reemplazó por límite de intentos por IP y validación en el servidor.
Carrito multi-producto, pasarela de pago y costo de envío por zona	Se implementó un pedido directo por producto; el carrito y el pago quedan como trabajo futuro por tiempo.
Reseñas y notificaciones de estado	No se alcanzó en el semestre; el estado del pedido se consulta en el panel.
Fotos y datos reales de los integrantes; fotografías reales de turismo	Se usan marcadores de posición e ilustraciones SVG; se reemplazan copiando los archivos con el mismo nombre.

7. Bibliografía (APA)

Bootstrap Team. (s. f.). *Bootstrap documentation*. https://getbootstrap.com/docs/

Express. (s. f.). *Express: Node.js web application framework*. https://expressjs.com/

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Google. (s. f.). *Gemini API documentation*. https://ai.google.dev/gemini-api/docs

Meta Platforms, Inc. (s. f.). *Documentación oficial de React*. https://react.dev/learn

Multer. (s. f.). *Multer: Middleware for handling multipart/form-data*. https://github.com/expressjs/multer

OpenJS Foundation. (s. f.). *Node.js documentation*. https://nodejs.org/docs/latest/api/

Oracle Corporation. (s. f.). *MySQL reference manual*. https://dev.mysql.com/doc/

React Bootstrap. (s. f.). *React Bootstrap documentation*. https://react-bootstrap.github.io/

Remix Software. (s. f.). *React Router documentation*. https://reactrouter.com/

Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum*. ScrumGuides.org.

8. Conclusiones

- Separar el proyecto en frontEnd y backEnd, y el backend en rutas y controladores, hizo el código más fácil de mantener y de repartir entre el equipo.
- Generar la base de datos y los datos iniciales desde el código elimina pasos manuales y reduce errores al desplegar en el equipo del profesor.
- Una fábrica de CRUD configurable permitió dar mantenimiento completo a todas las tablas sin repetir código.
- Integrar Gemini desde el servidor, con el catálogo real y validando su respuesta, aporta valor al usuario sin exponer la clave ni confiar ciegamente en el modelo.
- Servir los medios con express.static resolvió los problemas de multimedia de la Fase I.

9. Recomendaciones

- No subir nunca el archivo .env ni la clave de Gemini a GitHub; si se expone, revocarla y crear otra.
- Cambiar las contraseñas de prueba (Admin y Cliente) y el JWT_SECRET antes de un despliegue real.
- Agregar carrito, pasarela de pago y cálculo de envío por zona en una siguiente fase.
- Añadir pruebas automáticas de la API y paginación en las tablas del CRUD.
- Reemplazar las ilustraciones y los medios de demostración por material propio del equipo.

ANEXO 1. API estándar utilizada

API REST

El backend es una **API REST**: expone recursos mediante URL y los métodos HTTP estándar (GET consultar, POST crear, PUT modificar, DELETE eliminar) e intercambia datos en formato JSON sin guardar estado en el servidor (la sesión viaja en un token). Es la API con la que el frontend en React consume el servidor.

Ruta	Función
POST /api/auth/login · /registro	Iniciar sesión y crear cuenta
GET /api/categorias · /productos	Catálogo público (filtros ?cat= y ?q=)
POST /api/pedidos · GET /api/pedidos	Pedidos del cliente
GET/POST/PUT/DELETE /api/admin/:recurso	CRUD de las tablas (solo admin)
POST/GET/DELETE /api/archivos · /:id/descargar	Subir y bajar archivos
POST /api/ia/asesor	Asesor de IA (Gemini)

API de Google Gemini

El asesor consume la API REST de **Gemini** (`generativelanguage.googleapis.com`, método `generateContent`). El servidor envía el catálogo real, los datos del cliente y, si existe, la foto codificada en base64, y exige una respuesta en JSON. La clave se envía en el encabezado `x-goog-api-key` y se guarda únicamente en el archivo `.env` del servidor.